# Day 126 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 19, Day 1 reference

Watch: Autoencoders - Explained (DataMListic) | Autoencoder Explained - Deep Neural Networks (AIEngineering)

`day126_autoencoders.py`, segment by segment, in the order `main()` uses things. The MNIST loader, the dense network, its backpropagation, the two losses, Adam and the training loop are new; Day 123's PCA and Day 119's k-means are imported unchanged. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

**MATH BOX LEGEND**

**Symbols:** net = a list of (W, b) layers · A = every layer's output, A[0] the input · acts = each layer's activation · L = logits · dout = the gradient arriving from above

## Segment 0 — Setup

### Imports, threads and MNIST

**CODE**

```
import os
os.environ.setdefault("OPENBLAS_NUM_THREADS", "4")
import contextlib
import io
import sys
import time
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, os.path.join(HERE, "..", "week-18-unsupervised-learning-i"))
with contextlib.redirect_stdout(io.StringIO()):
    import day119_exercise as d119
    import day123_exercise as d123
CODE_ACTS = ["relu", "linear", "relu", "linear"]
def mnist():
    p = os.path.join(os.path.expanduser("~"), "scikit_learn_data", "mnist_784_uint8.npz")
    if not os.path.exists(p):
        from sklearn.datasets import fetch_openml
        d = fetch_openml("mnist_784", version=1, as_frame=False, parser="liac-arff")
        np.savez_compressed(p, X=d.data.astype(np.uint8), y=d.target.astype(np.uint8))
    z = np.load(p)
    return (z["X"] / 255.0).astype(np.float32), z["y"].astype(int)
```

- `os.environ.setdefault("OPENBLAS_NUM_THREADS", "4")` — must run before NumPy is imported; setdefault leaves a value the user already set.
- `np.savez_compressed(p, X=..., y=...)` — MNIST as whole numbers 0-255 in one small file: a second load takes under a second.
- `z["X"] / 255.0).astype(np.float32)` — pixels scaled to [0, 1]; float32 halves memory and was about 3 times faster here.

## Segment 1 — The network

### init_net, forward and backward

**MATH**

$$A_l = act(A_{l-1} W_l + b_l); \quad dW_l = A_{l-1}^T dZ_l; \quad dA_{l-1} = dZ_l W_l^T$$

**CODE**

```
def init_net(sizes, seed):
    rng = np.random.RandomState(seed)
    return [((rng.randn(a, b) * np.sqrt(2.0 / a)).astype(np.float32), np.zeros(b, np.float32)) for
        a, b in zip(sizes[:-1], sizes[1:])]
def forward(net, X, acts):
    A = [X]
    for (W, b), act in zip(net, acts):
        Z = A[-1] @ W + b
        A.append(np.maximum(Z, 0) if act == "relu" else Z)
    return A
def backward(net, A, acts, dout):
    grads = [None] * len(net)
    for i in range(len(net) - 1, -1, -1):
        dZ = dout * (A[i + 1] > 0) if acts[i] == "relu" else dout
        grads[i] = (A[i].T @ dZ, dZ.sum(0))
        dout = dZ @ net[i][0].T
    return grads, dout
```

- `zip(sizes[:-1], sizes[1:])` — consecutive pairs (784, 256), (256, 32), ...: one layer each.
- `np.sqrt(2.0 / a)` — He initialisation: keeps the signal's size steady through ReLU layers.
- `dout * (A[i + 1] > 0)` — the ReLU's gradient: 1 where the unit was on, 0 where it was off.
- `dout = dZ @ net[i][0].T` — passes the gradient to the layer below; the final value is the gradient for the input itself.

### sigmoid and recon_loss

**MATH**

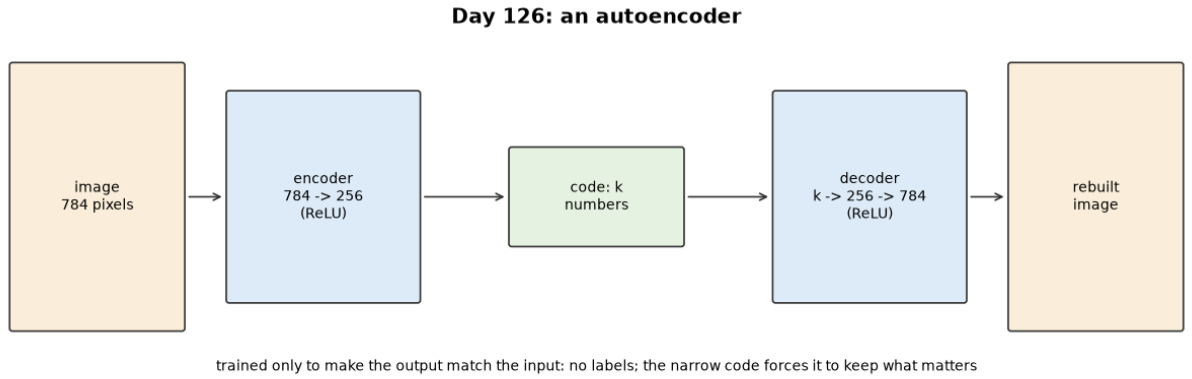
$$BCE = \Sigma_{pixels} [log(1 + e^l) - x\, l]; \quad \partial/\partial l = sigmoid(l) - x$$

**CODE**

```
def sigmoid(Z):
    return 0.5 * (1 + np.tanh(0.5 * Z))
def recon_loss(L, X, kind):
    n = len(X)
    if kind == "bce":
        return float((np.logaddexp(0, L) - X * L).sum() / n), (sigmoid(L) - X) / n
    return float(((L - X) ** 2).sum() / n), 2 * (L - X) / n
```

- `0.5 * (1 + np.tanh(0.5 * Z))` — the same as $1/(1 + e^{-Z})$, with no overflow for large negative z.
- `np.logaddexp(0, L) - X * L` — $\log(1 + e^L) - xL$: cross-entropy computed from logits.
- `(sigmoid(L) - X) / n` — its gradient: prediction minus target, per image.

SYSTEM DIAGRAM




*Encoder, code, decoder.*

# Segment 2 — Training

## Adam, train_ae and reconstruct

MATH

$$m \leftarrow 0.9m + 0.1g; \quad v \leftarrow 0.999v + 0.001g^2; \quad p \leftarrow p - lr \cdot (m/(1 - 0.9^t)) / (\sqrt{(v/(1 - 0.999^t))} + 10^{-8})$$

CODE

```
class Adam:
    def __init__(self, net, lr=1e-3):
        self.lr, self.t = lr, 0
        self.m = [(np.zeros_like(W), np.zeros_like(b)) for W, b in net]; self.v =
            [(np.zeros_like(W), np.zeros_like(b)) for W, b in net]
    def step(self, net, grads):
        self.t += 1; c1, c2 = 1 - 0.9 ** self.t, 1 - 0.999 ** self.t
        for (W, b), (gW, gb), m, v in zip(net, grads, self.m, self.v):
            for p, g, mm, vv in ((W, gW, m[0], v[0]), (b, gb, m[1], v[1])):
                mm *= 0.9; mm += 0.1 * g; vv *= 0.999; vv += 0.001 * g * g
                p -= self.lr * (mm / c1) / (np.sqrt(vv / c2) + 1e-8)
def train_ae(X, sizes, acts, kind="bce", epochs=20, batch=128, lr=1e-3, seed=0, Xval=None,
    noise=None):
    net = init_net(sizes, seed); opt = Adam(net, lr); rng = np.random.RandomState(seed + 1); hist
        = []
    for _ in range(epochs):
        o = rng.permutation(len(X))
        for s in range(0, len(X), batch):
            xb = X[o[s:s + batch]]; xin = xb if noise is None else noise(xb, rng)
            A = forward(net, xin, acts); _, d = recon_loss(A[-1], xb, kind)
            opt.step(net, backward(net, A, acts, d)[0])
        hist.append((recon_loss(forward(net, X[:2000], acts)[-1], X[:2000], kind)[0],
                     None if Xval is None else recon_loss(forward(net, Xval, acts)[-1], Xval,
                         kind)[0]))
    return net, hist
def reconstruct(net, X, acts, kind="bce"):
    L = forward(net, X, acts)[-1]
    return sigmoid(L) if kind == "bce" else L
```

- `mm *= 0.9; mm += 0.1 * g` — in-place updates: the stored averages change without new arrays.
- `p -= self.lr * ...` — -= on an array changes the network's own weights, which the list holds by reference.
- `xin = xb if noise is None else noise(xb, rng)` — a hook for Day 127: damage the input, keep the clean target.

- `hist.append((..., None if Xval is None else ...))` — a conditional expression inside a tuple: test loss only when test images are given.

## Segment 3 — The experiments

### demo_linear: comparing two subspaces

**CODE**

```
Q, _ = np.linalg.qr(net[1][0].T.astype(np.float64))
cos = np.linalg.svd(Q.T @ V[:, :32], compute_uv=False)
```

- `np.linalg.qr(...)` — an orthonormal basis for the decoder's 32 directions.
- `np.linalg.svd(Q.T @ V[:, :32], compute_uv=False)` — the singular values of $Q^TV$ are the cosines of the angles between the two subspaces (Day 124's SVD).

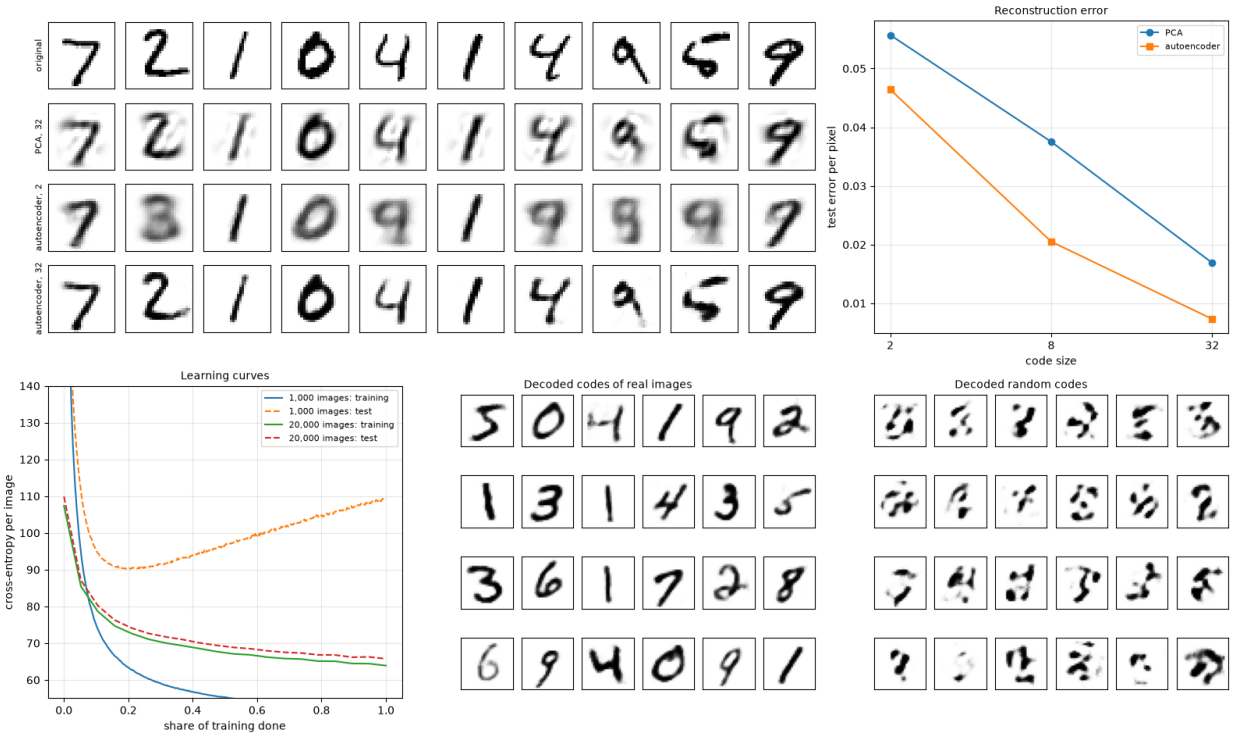
### demo_gaps: decoding with half the network

**CODE**

```
C = forward(net32, X[:20000], CODE_ACTS)[2]; mu, sd = C.mean(0), C.std(0); rng =
    np.random.RandomState(0)
real = sigmoid(forward(net32[2:], C[:500], CODE_ACTS[2:])[-1])
fake = sigmoid(forward(net32[2:], (mu + sd * rng.randn(500, 32)).astype(np.float32),
    CODE_ACTS[2:])[-1])
ref = X[20000:30000]
near = lambda R: np.sqrt(np.maximum(d119.sqdist(R.astype(np.float64), ref.astype(np.float64)),
    0).min(1) / 784)
```

- `forward(...)[2]` — the third entry of A: the code, after the input and the first hidden layer.
- `net32[2:], CODE_ACTS[2:]` — slicing the list of layers: the last two are the decoder, usable on their own.
- `mu + sd * rng.randn(500, 32)` — random codes with the real codes' mean and spread in each dimension.

**REAL GRAPH**




*Real graph from the script.*

## Quick Syntax Glossary — new or reinforced constructs

- `os.environ.setdefault(k, v)` — set an environment variable unless already set.
- `np.savez_compressed / np.load` — save and load several arrays in one file.
- `np.logaddexp(0, L)` — $\log(1 + e^{L})$ without overflow.
- `a *= 0.9` — change an array in place.
- `net[2:]` — a slice of a list of layers.
- `np.linalg.qr(A)` — an orthonormal basis for A's columns.

*End of Day 126 syntax reference. For the experiments and what they showed, see the Day 126 complete guide.*